

RANKTIER: EFFICIENT MIXTURE-OF-EXPERTS PREFILL ON AHEAD-OF-TIME COMPILED INFERENCE ACCELERATORS

Anonymous Authors¹

ABSTRACT

Mixture-of-experts (MoE) language models choose each token’s experts at run time, but server inference accelerators such as Qualcomm Cloud AI 100 execute ahead-of-time (AOT) compiled programs with static shapes and no data-dependent control flow. Each expert’s capacity is therefore fixed at compile time, and because some expert in some layer receives nearly every token of a prefill chunk, a program that never drops a token pads every expert to the chunk length; the exported combine adds a second cost that grows with the chunk, a dense per-lane accumulator. We present RankTier, which keeps the program static and moves the decisions it cannot compile into data. Each device ranks its experts by load at run time and binds fixed-capacity tiers to load ranks instead of expert identities, since the order statistics of per-device load are bounded and stable across workloads while the identities are not, and a token-owned combine sums each token’s expert rows where they were computed; both need only a sort and index gathers. On Qwen3-30B-A3B across the four SoCs of a Cloud AI 100 Ultra card, with the KV cache on device, RankTier prefills 128-, 256- and 512-token chunks $4.05\times$, $2.45\times$ and $2.94\times$ faster than the production compilation. With weight indirection and tile size matched, the rank-bound capacities make the ranked program $1.13\text{--}1.27\times$ faster; fixed-size blocks under the same compiler are $1.26\text{--}1.71\times$ slower; the calibrated capacities overflowed in none of 1872 evaluation chunks of eight workloads; and a program holding several capacity profiles recovers bit-exactly from a forced overflow. We separate the reasoning that transfers to other AOT targets from the hardware conditions under which it pays and from the compiler behavior that shaped our implementation.

1 INTRODUCTION

Mixture-of-experts (MoE) layers let a language model grow its parameter count without a matching growth in per-token compute: a router sends each token to a few of many experts, and only those experts run (Shazeer et al., 2017; Fedus et al., 2022). Recent open models use MoE layers throughout; Qwen3-30B-A3B routes each token to 8 of 128 experts in each of its 48 layers and activates 3.3B of its 30.5B parameters (Yang et al., 2025), and Mixtral and DeepSeek-V3 follow the same pattern (Jiang et al., 2024; DeepSeek-AI, 2024). During prefill, a serving system processes the prompt in chunks of T tokens (Agrawal et al., 2024), so each MoE layer sees $8T$ token-to-expert assignments whose distribution over experts is known only after its router runs. GPU systems absorb this variation with kernels that accept data-dependent shapes: block-sparse GEMMs that never drop a token (Gale et al., 2023), parallelism and pipelining chosen at run time (Hwang et al.,

2023), and all-to-all exchanges sized per batch (Rajbhandari et al., 2022).

Server inference accelerators take a different path. Qualcomm Cloud AI 100 (Chatha, 2021), like XLA-compiled TPU programs (Lepikhin et al., 2021) and the statically scheduled Groq TSP (Abts et al., 2020), runs a model as a program compiled ahead of time: tensor shapes, the partition of every operator over cores and devices, and the schedule are fixed before the first request, and on Cloud AI 100 the compiler rejects control flow that depends on data. Static planning is what lets the compiler orchestrate 16 cores per SoC and four SoCs per card, but for MoE it turns two run-time quantities into compile-time constants: how many token rows each expert computes, and how expert outputs flow back to their tokens. Both grow costly at long chunks, which prefill uses to amortize weight traffic, and they have two structural consequences.

(i) Padding sized by the worst chunk. Each expert occupies a lane of fixed capacity, and every lane computes its full capacity however few tokens it receives. Routing is skewed and workload-dependent: in our routing captures, one expert in some layer receives 256 of 256 and 508 of 512 tokens, and the experts most loaded in one workload

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

are rarely the most loaded in another (§3.1). A program that never drops a token therefore pads all 128 experts to T , $16\times$ the routed assignments. Calibrated per-expert capacities do not escape this: fitted to 200 prompts from all eight workloads, they still overflow on 12–36% of each workload’s held-out prompts, and they converge on T .

(ii) Reductions shaped by the compiler rather than by the tokens. By default the partitioner slices each expert’s down projection across the four SoCs, which costs 121.6 MiB of cross-SoC traffic per layer, and it returns expert outputs to tokens through a dense accumulator per stage that is zero-filled, updated between stages, reduced in serialized steps and summed across SoCs on a single SoC, spilling to DRAM from $T = 256$ on. Even with the slicing removed and the worst serialization rewritten, this combine path takes 51%, 64% and 68% of an MoE layer at $T = 128$, 256 and 512 in the production MXFP6 precision (§3.2). Expert-parallel compilation alone makes the full model 46% faster at $T = 128$ but 11–12.5% slower than production at 256 and 512 tokens, because the dense combine grows with T .

We present RankTier, a design for MoE prefill that keeps the program static and moves the decisions the compiler cannot make into data. It rests on one property that static programs share: a program cannot branch on data, but it can address by data. On Cloud AI 100, a weight gather indexed by a run-time tensor is lowered to indirect weight DMA and runs as fast as a static weight stream (§3.3). RankTier therefore fixes lanes and capacities at compile time and decides at run time which expert each lane serves. Each SoC counts the tokens routed to its 32 experts, sorts them, and gives the expert of rank r a lane in the tier that covers r . Capacities bound to ranks can be small because the order statistics of per-SoC load are stable across workloads even when expert identities are not: over 200 held-out prompts of eight workloads, the 16th most loaded expert of a SoC never received more than 12 of 128 tokens, while a fixed expert order needs up to 84–127 rows at every position (Figure 1). For the reduction, RankTier lets each device own its tokens: it gathers only the rows of each token’s experts that the device computed, sums them locally, and leaves one elementwise cross-device sum, which needs no root device.

We implement RankTier as graph transformations of the exported model and evaluate the full 48-layer Qwen3-30B-A3B with MXFP6 expert weights on the four SoCs of one Cloud AI 100 Ultra card, with the KV cache retained on device. RankTier prefills 128-, 256- and 512-token chunks in 122, 206 and 356 ms, $4.05\times$, $2.45\times$ and $2.94\times$ faster than the production compilation. The reduction redesign is the larger part and a prerequisite (2.06 – $2.38\times$ alone). With weight indirection and tile size matched, the rank-bound

capacities make the same ranked program 1.13, 1.13 and $1.27\times$ faster, and RankTier is 1.06 – $1.17\times$ faster than the best static program. Fixed-size blocks under the same compiler and combine are 1.26 – $1.71\times$ slower, because every block streams its expert’s weights. Capacities calibrated in advance overflowed in none of 1872 evaluation chunks of eight workloads, consecutive chunks through the on-device cache are bit-identical to the capacity- T program, and one program with several capacity profiles recovers bit-exactly from a forced overflow. The error against FP32 stays at the level of the production program.

We separate three things throughout. The algorithm needs only static shapes, a sort and gathers with run-time indices. Whether it pays depends on hardware, model and compiler: padded rows must cost time, and each device must hold many experts relative to k . The Cloud AI 100 compiler shaped the implementation and much of the speedup over production (§5, §7).

The paper makes three contributions:

- A method for MoE layers in static-shape programs that does not depend on the accelerator (§4): capacities bound to per-device load ranks, bounded by their order statistics, calibrated per tier and chosen against the target’s per-stage cost; exact recovery when a calibrated tier overflows; and a token-owned combine whose cost grows with the routed rows rather than with lanes $\times T$, together with how both compose with a compiler’s own tile-granular skipping.
- A realization on Cloud AI 100, with a characterization that separates what static shapes force on any AOT target from what this export and compiler add (§3, §5, Appendix B).
- An evaluation of the full model at three chunk lengths: a step-by-step ladder, an attribution of the capacity series with indirection and tile size matched, a fixed-size block baseline, held-out routing from eight workloads, and consecutive chunks with overflow detection and recovery (§6); and an account of the conditions under which each technique pays elsewhere (§7).

2 BACKGROUND AND RELATED WORK

2.1 MoE Inference

An MoE layer replaces the feed-forward block with E experts and a router. For a token with hidden state $x_t \in \mathbb{R}^d$, the router selects the set $\mathcal{K}_t$ of k experts with the largest gate values and returns

$$y_t = \sum_{e \in \mathcal{K}_t} w_{t,e} W_e^{\text{down}} (\text{SiLU}(W_e^{\text{gate}} x_t) \odot W_e^{\text{up}} x_t), \quad (1)$$

with normalized gate weights $w_{t,e}$. Qwen3-30B-A3B uses $E = 128$, $k = 8$, $d = 2048$ and an expert width of 768, so an expert’s three matrices hold 9 MiB in FP16. Implementations group assignments by expert, run each expert’s GEMMs over its gathered token rows, and return the weighted rows to their tokens; under expert parallelism the experts are spread over devices and dispatch and combine become cross-device exchanges (Lepikhin et al., 2021; Rajbhandari et al., 2022). Training systems bound each expert’s work by a capacity factor and drop the tokens beyond it (Lepikhin et al., 2021; Fedus et al., 2022); at inference, a dropped assignment changes the model’s output. Prefill processes the prompt in chunks of T tokens and writes their keys and values to the KV cache; decode then generates one token per step (Kwon et al., 2023; Agrawal et al., 2024). A chunk gives each layer $8T$ assignments, $2T$ per SoC on average when four SoCs hold 32 experts each.

2.2 MoE Systems

GPU systems handle the data-dependent shape of expert work directly: MegaBlocks computes experts as block-sparse GEMMs and never drops tokens (Gale et al., 2023), Tutel switches parallelism at run time (Hwang et al., 2023), and DeepSpeed-MoE, FasterMoE and Lina optimize or schedule the all-to-all exchange (Rajbhandari et al., 2022; He et al., 2022; Li et al., 2023). Placement studies exploit routing structure (Yao et al., 2024; Yu et al., 2025), Huang et al. (2024) replace static capacity with dynamic gating, and expert-choice routing fixes each expert’s tokens by construction but changes the model (Zhou et al., 2022). On Trainium, expert work is packed into fixed-size blocks whose weight loads are shared across an expert’s blocks and whose unused blocks are skipped (Amazon Web Services, 2026); we compare against blocks in §6.6. DynaMoE recompiles a static-shape program when capacities change during training (Kossmann et al., 2022); our full-model programs take 6–23 minutes to compile, so capacities cannot follow each chunk by recompilation.

2.3 AOT-Compiled Inference Accelerators

A Cloud AI 100 SoC has 16 AI cores and 136 GB/s of LPDDR4x bandwidth to up to 32 GB (Chatha, 2021; Qualcomm Technologies, 2025); each core has a scalar VLIW processor, a vector unit, a tensor unit with 4K FP16 MACs and 8 MB of software-managed vector memory. The Ultra card carries four SoCs under a shared 150 W limit.

The compiler takes an ONNX graph and one shape specialization per chunk length and produces a static program, partitioned over the cores of a SoC and, by multi-device tensor slicing, over the SoCs. Matrix weights can be MXFP6 (Rouhani et al., 2023), dequantized by the vector unit while the tensor unit computes. A KV cache stays on device

when each cache input can be paired with its updated output (*retained state*). An ONNX `if` with a non-constant condition is rejected; dynamic-shape compilers for CPUs and GPUs handle such workloads with run-time dispatch (Shen et al., 2021; Zhu et al., 2021), which these programs lack.

3 MOTIVATION

We study the MoE layers of Qwen3-30B-A3B compiled for the four SoCs of one Cloud AI 100 Ultra card (setup in §6.1). The exported graph (Qualcomm Innovation Center, 2024) runs a layer’s 128 experts as two stages of 64 *lanes*; a lane is one expert’s gate, up and down GEMMs over a fixed number of gathered token rows, its capacity C . The production program uses $C = T$ in both stages, the only capacity that can never drop a token. Three observations motivate RankTier: capacities attached to experts must cover the worst chunk, a combine that is dense in lanes grows with the chunk, and a static program can still address weights by data. The first follows from static shapes and the model’s routing and holds on any AOT target, the second holds for any lane-dense combine, and the third describes what this target offers.

3.1 Capacities Attached to Experts Cover the Worst Chunk

Without data-dependent control flow a lane cannot size its work to its tokens: the compiler skips only whole row tiles that hold no token (§6.3), so a lane computes its tokens rounded up to the tile and, unless it is one tile, reads all its weights. Capacity is work, and the question is which capacities a program may use without dropping tokens.

We captured the routing of all 48 layers for 50 prompts of 128 tokens from each of eight workloads, GSM8K, MMLU, SWE-bench Lite issues, HumanEval and its JavaScript version, Alpaca instructions, CNN/DailyMail articles and Chinese XNLI sentence pairs (Cobbe et al., 2021; Hendrycks et al., 2021; Jimenez et al., 2024; Chen et al., 2021; Muenighoff et al., 2024; Taori et al., 2023; Hermann et al., 2015; Conneau et al., 2018), and for chunks of 256 and 512 tokens from four of them (Appendix C), with the model’s FP32 layers on the CPU. The load is skewed: in some layer one expert receives 127 of 128, 256 of 256 and 508 of 512 tokens. The skew is also workload-specific: across workloads the 16 most loaded experts of a layer agree with a Jaccard index of 0.00–0.33, against 0.52–0.88 between the two halves of one workload, and the 64 most loaded agree at 0.29–0.70 (HumanEval and its JavaScript version, which share their problem statements, are the exception).

A program that attaches capacities to experts, fixing which lane serves which expert, must reserve at each lane the

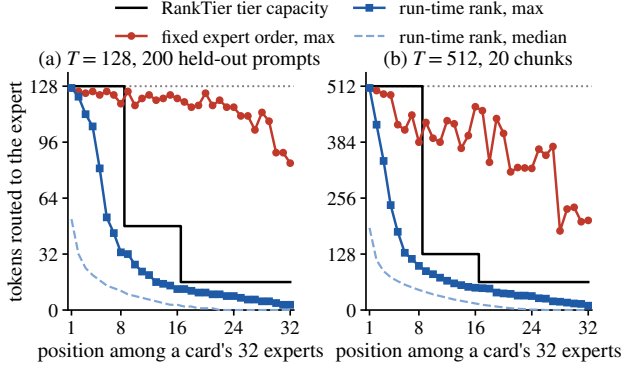


Figure 1. Rows a SoC must reserve for its expert at each position, as the largest load observed there. Red: experts in a fixed order, ranked by their mean load on calibration prompts (a: the first half of each workload’s prompts, evaluated on the other half; b: leave-one-out over the chunks). Blue: experts ordered at run time by their load in the chunk itself. Black: the tier capacities RankTier uses. Dotted: capacity T .

largest load that its expert can receive. Figure 1 shows the cost. Ordering each SoC’s 32 experts by their mean load on calibration prompts and reading the held-out loads at each position of that order, the maximum is 84–127 of 128 rows at every position at $T = 128$: a drop-free fixed layout needs 91% of the rows of padding everything to T (73% at $T = 512$). Fitting stage capacities to calibration data does not rescue a fixed layout: with hot and cold stages fitted to 200 prompts from all eight workloads, 12–36% of each workload’s held-out prompts still overflow in some layer, and the fitted capacities approach the chunk length, 122 and 105 of 128 rows in the median layer.

Padding is not free once T grows. In a one-layer replay, the tensor-unit time of a SoC’s 16 most loaded experts is 44, 310 and 921 μ s per core at capacities 92, 158 and 296, a $21\times$ increase for $3.2\times$ the rows, while their weight stream stays constant. In an instrumented profile of the full model at $T = 512$, the stage that holds each SoC’s 16 most loaded experts at capacity T takes a quarter of every layer (2.35 of 9.24 ms), and even after the reduction redesign of §4.5, capacity- T padding makes latency grow in proportion to the chunk (131.7, 258.9 and 498.4 ms at $T = 128$, 256 and 512; Table 2), so a longer chunk gains nothing per token.

3.2 A Lane-Dense Combine Grows with the Chunk

The combine of Eq. (1), expert rows back to their tokens, is where the export and its default compilation cost most. The exported combine scatters each stage’s weighted expert rows into an accumulator of shape $[64, T, d]$, updates it between the two stages, reduces it over each SoC’s 16 lanes and sums it over the four SoCs on one SoC; every part scales with lanes $\times T$, none with the routed rows. The compiler’s default partitioning adds an all-gather and a re-

Table 1. Device time of one MoE layer with the dense combine, split into the combine path (the accumulator’s gap between the stages and SoC 0’s reduction tail) and everything else. MXFP6, expert-parallel compilation and the dense-path rewrites of Appendix B.3, one-layer replays with stage capacities fitted to the input, instrumented.

T	Layer, ms	Combine path, ms	Rest, ms	Share
128	2.34	1.19	1.15	51%
256	4.35	2.78	1.57	64%
512	8.62	5.84	2.78	68%

shard of every expert’s down projection across the SoCs, 121.6 MiB of cross-SoC traffic per layer (Appendix B).

Table 1 measures the combine path after expert-parallel compilation and after the rewrites of Appendix B.3, which remove its most serialized parts: it still takes 51% of an MoE layer at $T = 128$ and 68% at 512. On the full model, compiling experts whole on each SoC with the KV cache kept on device (§5) reduces prefill latency from 494.6 to 267.1 ms at $T = 128$ but increases it from 505.3 to 568.6 ms at 256 and from 1049.1 to 1164.1 ms at 512 (Table 2), because its dense combine grows with T .

3.3 A Static Program Can Address by Data

What the compiler withholds is control flow; what it offers is indexing. Replacing the three weight constants of a layer’s first stage by `Gather(bank, idx)`, with the 64-entry index arriving at run time, leaves a one-layer replay at 2.796 ms, the same as with static weights, and bit-identical: the gather becomes an indirect weight DMA of the MXFP6 bytes. On the full model it costs 0% at $T = 256$ and 6% at 512 once tile sizes are tuned (§6.3); other gather forms are $1.7\text{--}3.7\times$ slower (Appendix B). The compiler also skips work at run time in one way of its own, again without control flow: it predicates each row tile of a lane on whether the tile holds a routed token (§4.6).

4 RANKTIER

Section 3 leads to three requirements. First, capacity must follow each chunk’s load without control flow, so the program must choose at run time which expert a lane serves and bind capacities to a quantity that is stable across chunks. Second, the combine must scale with the routed rows rather than with lanes $\times T$, and no device may root it. Third, both must be static graphs built only from operations that ahead-of-time targets provide: reductions, a sort, gathers with run-time indices and one cross-device sum. This section states the method in those terms, independent of the accelerator; §5 summarizes how the Cloud AI 100 compiler shaped its realization, and §7 separates what transfers.

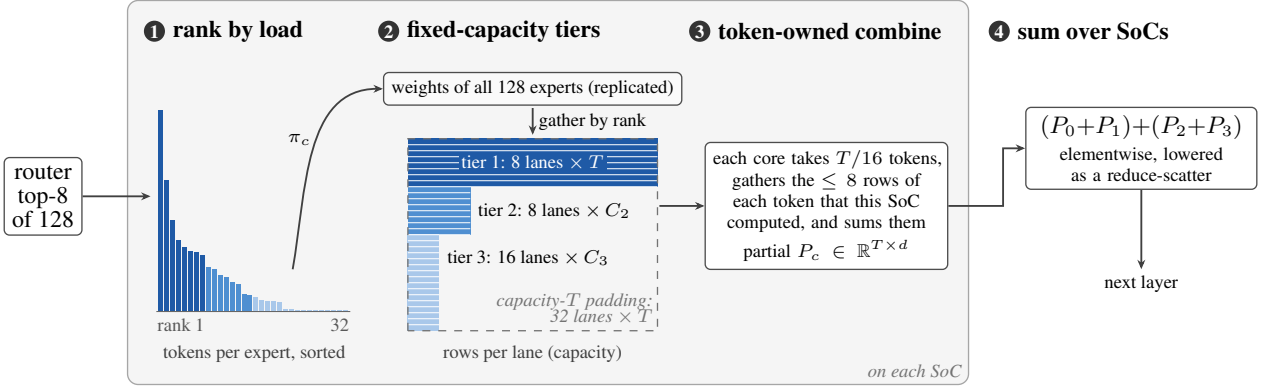


Figure 2. One MoE layer under RankTier. (1) Each SoC counts the tokens routed to its 32 experts and sorts them (bars: SoC 2 in layer 20 of the held-out 512-token chunk; colors mark the tiers). (2) Three tiers of lanes with compile-time capacities ($C_2 = 128$, $C_3 = 64$ at $T = 512$) serve the experts by rank; their weights are gathered from a bank of all 128 experts through the rank permutation π_c . The filled area is the padded work, against the dashed capacity- T layout. (3) Each core gathers and sums the rows of its own tokens. (4) One elementwise sum over SoCs completes the layer.

4.1 A Static Model of an MoE Layer

A device c owns a fixed set $\mathcal{E}_c$ of experts, 32 of the 128 on each SoC here. A static program computes them in *lanes*: a lane is one expert’s three GEMMs over C token rows gathered through a token table, and lanes of equal capacity that run together form a stage. Shapes, lanes and capacities are fixed at compile time; what may depend on the routing is data: which expert a lane serves, which tokens fill its rows and which weights it gathers. A chunk routes n_e of its kT assignments to expert e and $N_c = \sum_{e \in \mathcal{E}_c} n_e$ to device c , and a lane that serves e drops tokens unless $n_e \leq C$. A lane’s cost has a weight part, its expert’s matrices streamed from memory, and a row part that grows with the rows it computes: all C of them without control flow, or $R \lceil n_e/R \rceil$ on a target that predicates row tiles of R rows (§4.6). Either way the capacity bounds a lane’s work and must be fixed before the routing is known; the method chooses capacities that keep this bound small without dropping a token, and a combine whose cost does not depend on them.

4.2 Rank-Bound Capacities

After the router, each device counts its experts’ tokens and sorts the counts, which gives a permutation π_c from rank to expert with $n_{\pi_c(1)} \geq n_{\pi_c(2)} \geq \dots$. The device’s lanes form tiers: tier i is a stage of L_i lanes at capacity C_i that covers ranks $a_i, \dots, a_i + L_i - 1$, with $\sum_i L_i = |\mathcal{E}_c|$, and lane j of tier i serves expert $\pi_c(a_i + j)$. The lane gathers that expert’s weights through π_c and its token table from tables that the routing computes for all experts in their fixed order (Figure 2). Every expert keeps exactly one lane, so a tiering changes only the padding: the same experts compute the same tokens, and as long as every expert fits its lane, the output equals that of any other tiering, capacity T everywhere included (bit-identical in our implementation).

A device provisions $\sum_i L_i C_i$ rows per layer against $|\mathcal{E}_c| T$.

Why ranks. The order statistics of per-device load are bounded for a structural reason. The r most loaded experts of device c each hold at least $n_{\pi_c(r)}$ assignments, and together at most N_c , so

$$n_{\pi_c(r)} \leq N_c/r, \quad N_c \leq \min(k, |\mathcal{E}_c|) T. \quad (2)$$

N_c sums over all of the device’s experts, $kT|\mathcal{E}_c|/E = 2T$ on average here and at most $1.9\times$ that in our 256- and 512-token captures, so every rank beyond the first few is bounded well below T whatever the workload. A capacity attached to an expert must instead cover the worst case of that one expert, which approaches T because routing is skewed and which experts are hot changes with the workload (§3.1). Eq. (2) holds for any routing; that the per-rank maxima are also stable across workloads, while expert identities are not, is a property of the model, which we measured on held-out documents of eight workloads (Figure 1, §6.8).

4.3 Calibrating and Choosing Tiers

Capacities. The first tier has capacity T , since rank 1 can receive every token. For $i > 1$, C_i is the largest load observed at rank a_i , the tier’s first rank, over calibration chunks, times a headroom h , rounded up to the target’s row granularity g and capped at T ; since ranks are sorted, rank a_i bounds the whole tier. We calibrate with $h = 1.28$ and $g = 16$ on routing captures of the eight workloads: the 128-token prompts of §3.1, 56 chunks of 256 tokens and 20 of 512.

Structure. The number and widths of the tiers trade padded rows against per-stage cost. Rows alone favor many tiers: under our calibration three tiers need 65–67% of the rows of a two-tier split at T and $T/8$, four 52–58% and five

Algorithm 1 Tier selection for one device

Require: per-rank calibrated maxima m_r ; chunk length T ; headroom h ; row granularity g ; allowed widths $\mathcal{W}$; per-stage cost σ in rows

- 1: **for** each $(L_1, \dots, L_q)$ with $L_i \in \mathcal{W}$ and $\sum_i L_i = |\mathcal{E}_c|$ **do**
- 2: $a_1 \leftarrow 1$; $a_{i+1} \leftarrow a_i + L_i$; $C_1 \leftarrow T$
- 3: $C_i \leftarrow \min(T, g \lceil h m_{a_i} / g \rceil)$ for $i > 1$
- 4: $\text{cost} \leftarrow \sum_i L_i C_i + \sigma q$
- 5: **end for**
- 6: **return** the tiers (L_i, C_i) of least cost

48–56%. But each tier is a stage with a fixed cost of its own, and a target may restrict the widths a stage can take. RankTier therefore enumerates the tier structures whose widths come from the target’s allowed set and sum to $|\mathcal{E}_c|$, sizes each with the rule above, and keeps the structure of least cost (Algorithm 1), where the per-stage cost σ is measured on the target. On Cloud AI 100 the widths must divide 16 and stages run one after another (§5); comparing two, three and four tiers on the device gave three tiers of 8, 8 and 16 lanes per SoC at capacities T , C_2 and C_3 of 128/48/16, 256/64/32 and 512/128/64 for $T = 128, 256$ and 512, that is 12–13 T rows per SoC against 18 T for two tiers and 32 T for capacity- T padding.

4.4 Overflow Detection and Recovery

The lower tiers are calibrated, not dropless, so a deployment must detect overflow and recover from it exactly. Detection is free: the program already computes every lane’s token count, and the host compares it with the lane’s capacity after each chunk. Recovery reruns the chunk on a capacity- T variant at the same positions. Prefill writes the KV cache only at the chunk’s own positions, so the re-run overwrites every entry the failed attempt wrote, later chunks see only the rerun’s cache, and the result is that of an overflow-free run. The variant can be a second program kept resident or a second profile of the same program: tier capacities read from the lengths of small row inputs let the compiler specialize one program per profile over the same weights and cache. With an overflow probability p per chunk, the expected latency is $t_{\text{tier}} + p t_T$ for tiered and capacity- T latencies t_{tier} and t_T , so what matters is that the tiered program itself stays fast, not the fallback (§6.7).

4.5 Token-Owned Combine

The combine returns each token’s k weighted expert rows to the token and sums them. The exported graph formulates it per lane: each lane’s rows are scattered into an accumulator of shape $[\text{lanes}, T, d]$, which is then reduced over lanes, so its cost grows with $\text{lanes} \times T$ whatever the

Algorithm 2 Token-owned combine on SoC c

Require: top- k experts $\mathcal{K}[t, 1..k]$ of each token t ; lane map $\ell(e) \mapsto (s, c', l)$ (stage or tier, SoC, lane); slot $[e, t]$, row of token t in expert e ’s input; weighted outputs $O_s \in \mathbb{R}^{L_s C_s \times d}$ of SoC c

- 1: **for** all $t \leq T, j \leq k$ **in parallel do**
- 2: $e \leftarrow \mathcal{K}[t, j]$; $(s, c', l) \leftarrow \ell(e)$
- 3: $\text{idx}_{s'}[t, j] \leftarrow 0$; $m_{s'}[t, j] \leftarrow 0$ for every s' {masked}
- 4: **if** $c' = c$ **then**
- 5: $\text{idx}_s[t, j] \leftarrow l C_s + \text{slot}[e, t]$; $m_s[t, j] \leftarrow 1$
- 6: **end if**
- 7: **end for**
- 8: **for** each core q , owning tokens $t \in \text{tile } q$ **do**
- 9: $P_c[t] \leftarrow \sum_s \sum_j m_s[t, j] O_s[\text{idx}_s[t, j]]$ { $\leq k$ rows}
- 10: **end for**
- 11: $y \leftarrow (P_0 + P_1) + (P_2 + P_3)$ {elementwise, over SoCs}

routing. RankTier formulates it per token (Algorithm 2). The router’s top- k output already lists each token’s experts; with integer arithmetic on the lane layout and the per-expert slot tables that the routing computes anyway, every (token, j) pair maps to the tier and row of the device’s compact expert outputs that hold its result, or to a masked entry when the expert lives on another device. One gather per tier fetches at most k rows per token, their weighted sum is the device’s $[T, d]$ partial P_c , and one elementwise sum over the devices completes the layer.

Cost. Per device the gathers read at most kT rows of d elements, against LT rows of the dense accumulator for L lanes, and the cross-device traffic is one $[T, d]$ partial per device; split by token tiles, the final sum is a reduce-scatter with no root device. Each core owns a tile of tokens, so the per-device combine needs no cross-core traffic.

Why it comes first. Tiers of different widths cannot share one dense $[\text{lanes}, T, d]$ accumulator, so under the lane-dense combine each width needs its own, and the rows the tiers save are spent on accumulators that grow with T . The token-owned combine does not depend on the tiering at all, which is why RankTier fixes the combine before the capacities (§6.3).

4.6 Capacities and Tile-Granular Skipping

A compiler may already skip padded work at run time without control flow. Cloud AI 100 lowers the export’s mask of empty slots, $\text{Where}(r < n_e, y, 0)$, into a predicate per row tile of a lane: the tile’s gather, GEMMs and SiLU run only if the tile holds a routed token, so a lane computes $R \lceil n_e / R \rceil$ rows for a tile of R rows, and an empty expert’s weights are skipped only when its lane is a single tile (Appendix G). The tile follows the compiler’s maximum tile

size, from a whole 512-row lane down to 128 rows. Such skipping is coarse, and it does not shrink what a capacity provisions: token tables, buffers and the weight reads of multi-tile lanes stay at capacity. Rank-bound capacities compose with it. They set the lane size, and with it the tile, per rank: the low ranks get lanes that are single tiles and skip empty experts entirely, while the top ranks keep capacity T and tiles of their own. We therefore compare every program both at the compiler’s default tile size and at its best one (§6.3).

5 REALIZATION ON CLOUD AI 100

We realize RankTier as ONNX graph transformations of the Qwen3-30B-A3B graph that the vendor’s QEfficient library exports for Cloud AI 100 (Qualcomm Innovation Center, 2024), about 1,100 lines of Python with one pass per step of the evaluation’s ladder (§6.1). The compiler shaped five choices, detailed in Appendix B. (i) *Expert parallelism with an on-device cache*: forbidding the partitioner to cut weights across SoCs puts whole experts on each SoC, one lane per core, and cuts the layer’s cross-SoC traffic from 121.6 to 1.5 MiB, but it also stops the KV cache from staying on device; head-parallel attention, one KV-head group per SoC, restores it. (ii) *One gathered bank*: only a gather from one constant holding all 128 experts stays as fast as static weights, so the bank is replicated, 22.5 GiB per SoC instead of 6.7 GiB. (iii) *Tier widths*: a stage maps evenly onto a SoC’s cores only when its lanes divide 16, and the compiler runs a layer’s stages one after another, so widths come from $\{1, 2, 4, 8, 16\}$ and every tier adds a sequential window. (iv) *The sort off the critical path*: the routing tables are computed for all experts in their fixed order, and only a row permutation and the weight gathers wait for the sort; the maximum tile size is chosen per program and chunk length (Appendix E). (v) *Lowering repairs*: three rewrites remove serialized work from the exported dense combine, and the evaluation keeps them as a step of their own. Each chunk length is its own program, with sequence length T , context $2T$, MXFP6 matrix weights and FP16 activations (Appendix A).

6 EVALUATION

We ask how much RankTier speeds up the full model (§6.2), what the capacities contribute once indirection and tiling are matched (§6.3), what each part of the reduction and of the padding design contributes (§6.4, §6.5), how ranked lanes compare with fixed-size blocks (§6.6), whether the programs hold beyond the first chunk and under over-flow (§6.7), whether the calibration generalizes (§6.8), and whether the output is preserved (§6.9).

6.1 Experimental Setup

Hardware and model. One Cloud AI 100 Ultra card, four SoCs with 16 cores and 32 GB each, platform SDK 1.21.6. Qwen3-30B-A3B, all 48 layers, batch 1, MXFP6 matrix weights and FP16 activations as in the production deployment. Every program keeps its KV cache on device.

Programs. Seven programs, each adding one step to the one before it (Table 2): (1) the production compilation of the exported graph, with default partitioning, the dense combine and capacity T in both stages; (2) expert-parallel compilation with head-parallel attention, which keeps the export’s combine and capacity T and is our *naive* T/T baseline; (3) the dense-path rewrites; (4) the token-owned combine; (5) the elementwise final sum; (6) the run-time rank sort with two tiers at T and $T/8$; (7) RankTier’s three tiers.

Prompts. At $T = 128$ the first chunk of a GSM8K chat prompt; at 256 and 512 held-out chunks built from GSM8K test questions outside the calibration range. Latency depends on a chunk only through its routing, and on a single layer it varied by about 1% across 200 prompts of four workloads.

Protocol. Host latency of one chunk, the median of three rounds of 20 inferences after warm-up. All programs of one chunk length run back to back in one session, with anchor programs repeated before and after; steps 6 and 7 ran in a second session and are chained to the first through step 5, which ran in both and agreed within 0.13%. Rounds agree within 0.6%. The attribution programs of §6.3 ran in sessions of two or three, each timed twice in mirrored order, every session holding an anchor; a program + session fit of log latency combines them (largest residual 0.34%). For accuracy we compare the last-position logits with an FP32 CPU run of the same chunk, check the next-token prediction, and read back every lane’s token count.

6.2 End-to-End Results

Table 2 gives the results (Figure 3 in Appendix F plots them). RankTier prefills 128-, 256- and 512-token chunks in 122.2, 206.2 and 356.3 ms, $4.05\times$, $2.45\times$ and $2.94\times$ faster than the production compilation, and $2.19\times$, $2.76\times$ and $3.27\times$ faster than naive T/T . In the order of the ladder, the reduction steps contribute 2.06 – $2.37\times$ and the padding steps after them 1.06 – $1.38\times$, both growing with T ; §6.3 separates the two series. Only with both does a longer chunk pay: per-token latency falls from 0.95 to 0.70 ms between $T = 128$ and 512 under RankTier, stays at 0.97–1.03 ms with the token-owned combine at capacity T , and rises from 2.09 to 2.27 ms under naive T/T .

Table 2. Prefill latency of one chunk (ms) for the seven programs, and the change against the program above it.

Program	$T = 128$	$T = 256$	$T = 512$
1 Production	494.6	505.3	1049.1
2 + expert/head par.	267.1 –46.0%	568.6 +12.5%	1164.1 +11.0%
3 + rewrites	158.4 –40.7%	355.9 –37.4%	729.1 –37.4%
4 + token-owned	131.7 –16.9%	258.9 –27.3%	498.4 –31.6%
5 + elementwise sum	129.5 –1.6%	252.1 –2.6%	491.1 –1.5%
6 + rank sort	126.3 –2.5%	215.2 –14.6%	421.6 –14.2%
7 + tiers (RankTier)	122.2 –3.2%	206.2 –4.2%	356.3 –15.5%
Reductions, 2→5	2.06×	2.26×	2.37×
Padding, 5→7	1.06×	1.22×	1.38×
vs. naive T/T , 2→7	2.19×	2.76×	3.27×
vs. production	4.05×	2.45×	2.94×
Tokens/s, 1 / 7	259 / 1047	507 / 1241	488 / 1437

Table 3. Capacity attribution on the full model (ms per chunk). Default: every program at the default tile size; tuned: each at its best size (KiB, A–E: def/1024/512/512/512 at $T = 128$, 512/512/512/512/1024 at 256, 512/1024/1024/1024/1024 at 512). Rows: expert rows provisioned per SoC.

	$T = 128$		$T = 256$		$T = 512$	
	default	tuned	default	tuned	default	tuned
A static, T	130.3	130.3	256.6	236.0	497.9	417.6
B gather, T	130.3	126.2	249.8	235.1	492.6	442.3
C rank, T	147.5	138.1	270.2	233.6	505.0	450.8
D rank, 2 tiers	133.8	125.7	243.8	210.9	431.1	415.3
E RankTier	128.0	122.5	208.1	207.1	376.1	355.8
C/E capacities	1.15×	1.13×	1.30×	1.13×	1.34×	1.27×
A/E vs. static	1.02×	1.06×	1.23×	1.14×	1.32×	1.17×
Rows, A–C / E	4096 / 1664		8192 / 3072		16384 / 6144	

6.3 Attributing the Padding Series

The ladder changes weight access (static per-lane constants against gathers from the replicated bank), the assignment of experts to lanes and the capacities between steps 5 and 7, and it compares programs at different tile sizes. Table 3 separates them with five programs that share the reduction series: (A) fixed order, capacity T , static weights; (B) the same order through the gathers, as run-time data; (C) the run-time rank at capacity T ; (D) the two tiers of step 6; (E) RankTier. All give bit-identical logits. Each is timed at the default tile size (–size-split-granularity) and at its best of three sizes on two-layer cuts.

Capacities after paying for dynamic selection. C and E both rank and gather; only the capacities differ. RankTier is 1.13, 1.13 and 1.27× faster at $T = 128$, 256 and 512 with tuned tiles (1.15–1.34× at the default), and 1.06–1.17× faster than the best static program, which cannot follow the ranking.

Why the tile size matters. The tile size sets the tiles of §4.6: a 512-row lane is one tile at the default, two at

1024 KiB and four at 512 KiB, where the static program computes 3.5–3.8k of 16.4k rows per SoC instead of 13.8–15.4k and becomes weight-bound. Gathered GEMMs skip the same way, but at 512 KiB the compiler spills their activations (95–97 MiB per SoC and layer), so they run at 1024 KiB with twice the rows: the 6% of A→B at $T = 512$, a property of this compiler rather than of indirection (none at $T \leq 256$). Capacities make the tile follow the rank: eight lanes at capacity T spend 0.4 ms of tensor-unit time per lane where sixteen spend 1.3–1.7 ms, and RankTier’s 64-row lanes are single tiles. The sort costs 0–2% at $T \geq 256$ and 9% at 128.

The order of the series. Under the export’s combine the same tiers make the program slower (0.69–0.85× of naive T/T), since each tier width needs its own dense accumulator; the reduction series, 2.06–2.38× alone, is the prerequisite.

6.4 Reduction

Expert parallelism alone is not enough. Step 2 removes the down projection’s slicing and is 46% faster than production at $T = 128$. At 256 and 512 tokens it is 12.5% and 11.0% slower, because the dense combine that remains grows with T (Table 1); the head-parallel attention in this step only restores the on-device cache.

Representation as well as lowering. The dense-path rewrites save 37–41% at every chunk length by repairing the export’s lowering of the dense combine; replacing the dense representation by the token-owned combine saves another 16.9%, 27.3% and 31.6%, growing with T as the accumulator it removes does. In instrumented two-layer profiles at $T = 512$ it cuts the time after the last expert GEMM from 4.7–5.5 to 1.2–1.4 ms per layer and the MoE’s cross-SoC traffic from 48 to 14 MiB. The elementwise final sum adds 1.5–2.6%.

6.5 Padding

The sort is cheap; the capacities pay. Step 6 of the ladder saves 2.5%, 14.6% and 14.2%, but it also changes weight access and tile size. With both matched (Table 3) the sort itself costs 0–2% at $T = 256$ and 512 but 9% at 128, where it sits on a short layer’s critical path (B→C), and the two tiers give 1.09–1.11× (C→D). The sort adds a dependency on every layer’s critical path, router to sort to weight gathers, which the tile sizes of Appendix B.4 keep short.

Three tiers, not more. Step 7 saves another 3.2%, 4.2% and 15.5%. At $T = 512$ the 16 lanes at capacity 512 take 2.2 ms per layer; as eight lanes at 512 and eight at 128, the same ranks take 0.8–1.0 and 0.65–0.72 ms (Figure 4, Appendix J), since the stage is bound by the SoC’s memory traffic. A fourth tier removes more rows but adds a sequen-

tial window: on two layers, four tiers are 8.6% faster than two, three tiers 14.1%. At $T = 256$ the extra window absorbs most of the saving.

6.6 Fixed-Size Blocks Instead of Ranked Lanes

Block scheduling packs assignments into blocks of B rows, one expert per block, and provisions $\lceil kT/B \rceil + E - 1$ blocks to drop nothing (Amazon Web Services, 2026). We built it from RankTier’s own stage body, routing, gathers and combine: each SoC computes in the graph the blocks per expert, the expert and rows of every block slot, and the combine reads an assignment at row $M_e B + s$. Budgets per SoC: dropless for the worst local load of $8T$ assignments, $\min(\lceil 8T/B \rceil + 31, 32\lceil T/B \rceil)$; calibrated by RankTier’s rule; and, as a non-deployable diagnostic, the timed prompt’s active blocks. Outputs are bit-identical to RankTier.

The first block programs were $2.0\text{--}4.8\times$ slower because the reused stage body materialized the hidden states once per lane (192 MiB per SoC and layer at 96 lanes); a plain gather of the rows removed that (Table 9 in Appendix H) but does not help RankTier (16.4 against 16.0 ms). What remains is weight traffic: every block slot streams its expert’s MXFP6 weights (3.52 MiB), used or not, so the 64-row dropless program reads 337.5 MiB per SoC and layer against 95–106 MiB for RankTier, whose empty experts skip their reads. One weight load cannot serve consecutive blocks of an expert in this compiler; merging them into one lane with a static row count is a capacity, i.e. RankTier. At $T = 512$ the best calibrated block program is $1.26\times$ slower than RankTier, the best dropless one $1.51\times$, and even the trace-specialized one $1.12\times$, as its single stage runs four lanes per core on 12 of the 16 cores and spills; at $T = 256$ these are 1.39 , 1.59 and $1.13\times$, at 128 1.69 , 1.71 and $1.31\times$. Blocks lose on weight traffic, not rows: at $T = 512$ the trace-specialized programs and the calibrated ones with $B \leq 64$ provision fewer rows than RankTier.

6.7 Beyond the First Chunk, and Overflow

The programs keep the cache for $2T$ positions. A second chunk attends to the cache the first wrote: RankTier and the static capacity- T program give bit-identical logits for both chunks at $T = 128$, 256 and 512, matching the FP32 reference’s top five at positions $T - 1$ and $2T - 1$ and its next token except at two near-ties at $T = 128$ (0.38 and 0.58 logits apart), and the second chunk costs what the first does.

We tested the recovery of §4.4 with capacity profiles in one program. With an undersized profile forced on the second of four chunks of the full model, the overflow was detected in 45 (layer, lane) pairs from layer 0 on, the rejected logits were 0.04 relative L_2 off, and after the rerun all four chunks

were bit-identical to runs without overflow, at full capacity and on the static program; the same holds for an overflow in the first chunk. Capacity inputs and extra profiles make the tiered program 18% slower, however, so a static capacity- T program kept resident beside RankTier for replays may be the cheaper fallback.

6.8 Do Calibrated Capacities Generalize?

We split each of eight workloads into calibration and evaluation documents before cutting chunks and measured the routing of every chunk on the device. Calibrated without headroom, the rank rule reproduces RankTier’s capacities exactly at $T = 256$ (at 512 it gives a 112-row middle tier where RankTier uses 128, which overflows in 2 of 449 chunks; at 128 a 32-row one where RankTier uses 48). RankTier’s capacities overflowed in none of the 805, 618 and 449 evaluation chunks at $T = 128$, 256 and 512 and in no workload (30 chunks hold documents they were calibrated on), nor did leave-one-workload-out calibration. The same tiers bound to a fixed expert order need capacity T everywhere, and even calibrated per workload they need $1.4\text{--}2.7\times$ the rows and still overflow in up to 30% of chunks: at $T = 512$ the busiest expert at a SoC’s positions 9–16 or 17–32 exceeds 128 tokens in 30 or 23% of (chunk, layer, SoC) cases, because which experts are busy moves from chunk to chunk, while ranks 1, 9 and 17 never exceeded 509, 119 and 52 tokens. The bound $n_{(r)} \leq N/r$ holds with room (at most 0.97 of it).

6.9 Accuracy

Every program stays at the error level of the production program’s MXFP6 weights: the relative L_2 error of the last-position logits against FP32 is 0.170, 0.087 and 0.088 for RankTier at $T = 128$, 256 and 512, against 0.188, 0.089 and 0.095 for production (Appendix K). The programs differ from production only in FP16 summation order (head-parallel attention regroups the output projection; the token-owned combine reorders a token’s rows), and the attribution and block programs are bit-identical to RankTier. The next token matches FP32 except at 256 tokens, whose two leading FP32 logits are 0.58 apart. No lane exceeded its capacity.

6.10 Discussion and Limitations

Scope. The lower tiers are calibrated, not dropless: no evaluation chunk overflowed, but the deployed rate depends on model and workload. One model at batch 1; two consecutive chunks at every T and four at $T=128$, but no decode, which needs a decode graph that our T -specific rewrites do not provide; blocks under this compiler only. The SoCs share a 150 W limit under which most padded programs throttle at times (RankTier never does), so we compare pro-

Table 4. What transfers. For each part of RankTier: the reasoning that holds on any static-shape target, the conditions under which it pays, and the Cloud AI 100 compiler behavior that shaped our realization of it.

Part	Transferable reasoning	Pays when	Shaped here by the compiler
Rank-bound capacities	a lane serves a load rank within a device’s experts; the r -th largest of n assignments is at most n/r , and per-rank maxima were stable across workloads while identities were not; blocks stream weights per block	expert work is fixed before routing is known; padded rows cost time and the compiler skips them only in whole tiles; many experts per device	no control flow, so a sort and gathers; one gathered constant, replicated (22.5 GiB per SoC); tile size tuned per chunk length
Tier sizes	first tier at T ; lower tiers at h times the calibrated maximum of their first rank; the tier count trades rows against per-stage cost; overflows show in the counts	a stage’s fixed cost is small against the rows it removes	lanes per SoC must divide 16 and stages run in sequence, hence three tiers of 8, 8 and 16
Token-owned combine	sum each token’s $\leq k$ rows where they were computed, then one cross-device sum; cost grows with kT , not with lanes $\times T$; no root device	the baseline combine is dense in lanes (this export, GShard-style einsums); many lanes per device relative to k	rewrites of a zero read, a serial lane reduction and serial prefix sums (1.60 – $1.69\times$); gather without zero fill; work kept on the lane axis; elementwise final sum

grams within a session or across sessions through repeated anchors.

7 WHAT TRANSFERS

The speedups of §6 combine an algorithm, conditions of hardware and model that decide how much it pays, and compiler behavior that decided how we built it; we call a part transferable when the same reasoning applies on another target, even if the transformation and the speedup differ (Table 4).

The algorithm. Capacities fixed before routing is known should follow load ranks, the combine should be owned by tokens, and it must be fixed before the padding (§6.3). All three need only static shapes, a sort and gathers with run-time indices. The order-statistics bound holds for any routing; its stability across workloads is a property of the model, measured here for one.

When it pays. The techniques target programs that fix an expert’s work before routing is known; kernels that size expert work at run time have no padding to remove (Gale et al., 2023). Padded rows must cost time, which depends on the hardware and on the compiler: a 16-lane stage of ours streams its MXFP6 weights in about 0.6 ms, stays at that floor with 92 rows per lane and is bound by its tensor-unit time with 296, and the compiler skips padded rows only in whole tiles, coarser ones for gathered weights, so the capacities gain 1.13 – $1.27\times$ over the ranked program and 1.06 – $1.17\times$ over the best static one. Blocks need what whole-expert lanes do not, a weight load shared by an expert’s consecutive blocks or skipped for unused ones; where a stack provides both (Amazon Web Services, 2026) they may close the 1.12 – $1.71\times$ gap we measured. Both techniques need many experts per device relative to k (32

per SoC against $k = 8$ here; 8 experts with $k = 2$ on four devices would leave little to rank), gathers that cost about as much as a static weight stream, and, for the combine, a lane-dense baseline such as this export’s or GShard’s einsums (Lepikhin et al., 2021).

What the compiler shaped. The dense-path rewrites (1.60 – $1.69\times$) repair this export and compiler; the token-owned combine and the elementwise final sum add 1.22 – $1.48\times$ after them. The production baseline reflects the default partitioning: $1.85\times$ slower than naive T/T at $T = 128$ but 1.11 – $1.13\times$ faster at 256 and 512, so our speedups over production do not transfer, while the reasoning does. A scheduler that overlaps independent stages would favor four or five tiers.

8 CONCLUSION

Ahead-of-time compilation fixes how many rows each expert computes and how its outputs return to their tokens before routing is known. RankTier moves both decisions into data: capacities follow per-device load ranks, whose order statistics are bounded and stable when expert identities are not, and the combine is owned by tokens. On Qwen3-30B-A3B across four Cloud AI 100 SoCs, RankTier prefills 128-, 256- and 512-token chunks $4.05\times$, $2.45\times$ and $2.94\times$ faster than the production compilation; with in-direction and tiling matched, its capacities give 1.13 – $1.27\times$ and beat fixed-size blocks by 1.26 – $1.71\times$.

REFERENCES

Abts, D., Ross, J., Sparling, J., Wong-VanHaren, M., Baker, M., Hawkins, T., et al. Think fast: A tensor streaming processor (TSP) for accelerating deep learning workloads. In *Proceedings of the 47th Annual International*

- Symposium on Computer Architecture (ISCA), pp. 145–158, 2020.
- Agrawal, A., Kedia, N., Panwar, A., Mohan, J., Kwatra, N., Gulavani, B. S., Tumanov, A., and Ramjee, R. Tam- ing throughput-latency tradeoff in LLM inference with Sarathi-Serve. In *18th USENIX Symposium on Oper- ating Systems Design and Implementation (OSDI)*, pp. 117–134, 2024.
- Ainslie, J., Lee-Thorp, J., de Jong, M., Zemlyanskiy, Y., Lebrón, F., and Sanghai, S. GQA: Training generalized multi-query transformer models from multi-head check- points. In *Proceedings of the 2023 Conference on Empir- ical Methods in Natural Language Processing (EMNLP)*, pp. 4895–4901, 2023.
- Amazon Web Services. MoE inference archi- tecture deep dive, AWS Neuron SDK 2.30.0. https://awsdocs-neuron.readthedocs-hosted.com/en/v2.30.0/libraries/nxd-inference/developer_guides/moe-arch-deep-dive.html, 2026. Accessed September 30, 2026.
- Chatha, K. Qualcomm cloud AI 100: 12 TOPS/W scal- able, high performance and low latency deep learning inference accelerator. In *2021 IEEE Hot Chips 33 Sym- posium (HCS)*, pp. 1–19, 2021. doi: 10.1109/HCS52781.2021.9567417.
- Chen, M., Tworek, J., Jun, H., Yuan, Q., et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C., and Schulman, J. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- Conneau, A., Rinott, R., Lample, G., Williams, A., Bow- man, S., Schwenk, H., and Stoyanov, V. XNLI: Evalu- ating cross-lingual sentence representations. In *Proceed- ings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 2475–2485, 2018.
- DeepSeek-AI. DeepSeek-V3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.
- Fedus, W., Zoph, B., and Shazeer, N. Switch transformers: Scaling to trillion parameter models with simple and ef- ficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.
- Gale, T., Narayanan, D., Young, C., and Zaharia, M. MegaBlocks: Efficient sparse training with mixture-of- experts. In *Proceedings of Machine Learning and Sys- tems (MLSys)*, volume 5, 2023.
- He, J., Zhai, J., Antunes, T., Wang, H., Luo, F., Shi, S., and Li, Q. FasterMoE: Modeling and optimizing training of large-scale dynamic pre-trained models. In *Proceed- ings of the 27th ACM SIGPLAN Symposium on Princi- ples and Practice of Parallel Programming (PPoPP)*, pp. 120–134, 2022.
- Hendrycks, D., Burns, C., Basart, S., Zou, A., Mazeika, M., Song, D., and Steinhardt, J. Measuring massive mul- titask language understanding. In *International Confer- ence on Learning Representations (ICLR)*, 2021.
- Hermann, K. M., Kočiský, T., Grefenstette, E., Espeholt, L., Kay, W., Suleyman, M., and Blunsom, P. Teach- ing machines to read and comprehend. In *Advances in Neural Information Processing Systems (NeurIPS)*, vol- ume 28, 2015.
- Hillis, W. D. and Steele, Guy L., J. Data parallel algorithms. *Communications of the ACM*, 29(12):1170–1183, 1986.
- Huang, H., Ardalani, N., Sun, A., Ke, L., Lee, H.-H. S., Bhosale, S., Wu, C.-J., and Lee, B. Toward efficient inference for mixture of experts. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, 2024.
- Hwang, C., Cui, W., Xiong, Y., Yang, Z., Liu, Z., Hu, H., Wang, Z., Salas, R., Jose, J., Ram, P., Chau, J., Cheng, P., Yang, F., Yang, M., and Xiong, Y. Tutel: Adaptive mixture-of-experts at scale. In *Proceedings of Machine Learning and Systems (MLSys)*, volume 5, 2023.
- Jiang, A. Q., Sablayrolles, A., Roux, A., Mensch, A., Savary, B., Bamford, C., et al. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024.
- Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., and Narasimhan, K. SWE-bench: Can language mod- els resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.
- Kossmann, F., Jia, Z., and Aiken, A. Optimizing mixture of experts using dynamic recompilations. *arXiv preprint arXiv:2205.01848*, 2022.
- Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Sympo- sium on Operating Systems Principles (SOSP)*, pp. 611–626, 2023.

- Lepikhin, D., Lee, H., Xu, Y., Chen, D., Firat, O., Huang, Y., Krikun, M., Shazeer, N., and Chen, Z. GShard: Scaling giant models with conditional computation and automatic sharding. In *International Conference on Learning Representations (ICLR)*, 2021.
- Li, J., Jiang, Y., Zhu, Y., Wang, C., and Xu, H. Accelerating distributed MoE training and inference with Lina. In *2023 USENIX Annual Technical Conference (USENIX ATC)*, pp. 945–959, 2023.
- Muennighoff, N., Liu, Q., Zebaze, A., Zheng, Q., Hui, B., Zhuo, T. Y., Singh, S., Tang, X., von Werra, L., and Longpre, S. OctoPack: Instruction tuning code large language models. In *International Conference on Learning Representations (ICLR)*, 2024.
- Qualcomm Innovation Center. Efficient transformers library (QEfficient). <https://github.com/quic/efficient-transformers>, 2024.
- Qualcomm Technologies. Cloud AI 100 architecture. Qualcomm Cloud AI SDK documentation, <https://quic.github.io/cloud-ai-sdk-pages/latest/Getting-Started/Architecture/>, 2025.
- Rajbhandari, S., Li, C., Yao, Z., Zhang, M., Aminabadi, R. Y., Awan, A. A., Rasley, J., and He, Y. DeepSpeed-MoE: Advancing mixture-of-experts inference and training to power next-generation AI scale. In *Proceedings of the 39th International Conference on Machine Learning (ICML)*, volume 162 of *Proceedings of Machine Learning Research*, pp. 18332–18346, 2022.
- Rouhani, B. D., Zhao, R., More, A., Hall, M., Khodamoradi, A., Deng, S., Choudhary, D., Cornea, M., Dellinger, E., Denolf, K., et al. Microscaling data formats for deep learning. *arXiv preprint arXiv:2310.10537*, 2023.
- Shazeer, N., Mirhoseini, A., Maziarz, K., Davis, A., Le, Q., Hinton, G., and Dean, J. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations (ICLR)*, 2017.
- Shen, H., Roesch, J., Chen, Z., Chen, W., Wu, Y., Li, M., Sharma, V., Tatlock, Z., and Wang, Y. Nimble: Efficiently compiling dynamic neural networks for model inference. In *Proceedings of Machine Learning and Systems (MLSys)*, volume 3, pp. 208–222, 2021.
- Taori, R., Gulrajani, I., Zhang, T., Dubois, Y., Li, X., Guestrin, C., Liang, P., and Hashimoto, T. B. Stanford Alpaca: An instruction-following LLaMA model. https://github.com/tatsu-lab/stanford_alpaca, 2023.
- Yang, A. et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- Yao, J., Anthony, Q., Shafi, A., Subramoni, H., and Panda, D. K. Exploiting inter-layer expert affinity for accelerating mixture-of-experts model inference. *arXiv preprint arXiv:2401.08383*, 2024.
- Yu, Z., Guan, Y., Yu, Z., Zhou, C., Hu, Z., Pei, S., Kang, Y., Ding, Y., and Tsai, P.-A. Patterns behind chaos: Forecasting data movement for efficient large-scale MoE LLM inference. *arXiv preprint arXiv:2510.05497*, 2025.
- Zhou, Y., Lei, T., Liu, H., Du, N., Huang, Y., Zhao, V., Dai, A., Chen, Z., Le, Q., and Laudon, J. Mixture-of-experts with expert choice routing. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, 2022.
- Zhu, K., Zhao, W., Zheng, Z., Guo, T., Zhao, P., Bai, J., Yang, J., Liu, X., Diao, L., and Lin, W. DISC: A dynamic shape compiler for machine learning workloads. In *Proceedings of the 1st Workshop on Machine Learning and Systems (EuroMLSys)*, pp. 89–95, 2021.

A COMPILATION FLAGS AND PROGRAM SIZES

All programs are compiled with the command below, with a four-SoC partition configuration and one shape specialization per chunk length (sequence length T , context $2T$). Steps 2–7 add `-mdts-mos=1`; steps 6–7 also set the maximum tile size, 512 KiB for $T \leq 256$ and 1024 KiB for $T = 512$.

```
gaic-compile -aic-hw -aic-hw-version=ai100
-retained-state -convert-to-fp16
-mx6-matmul -aic-num-cores=16 -mos=1
-aic-enable-depth-first
-mdp-load-partition-config=<4 SoCs>
-network-specialization-config=<T, 2T>
[-mdts-mos=1] # steps 2-7
[-size-split-granularity=512|1024] # 6-7
```

Table 5 lists the program sizes.

Table 5. Program file, device memory per SoC and compile time.

Programs	File	Memory/SoC	Compile
Steps 1–5, static weights	25 GiB	6.6–7.1 GiB	6–12 min
Steps 6–7, gathered	88 GiB	22.4–22.5 GiB	12–23 min

B REALIZATION DETAILS ON CLOUD AI 100

This appendix records how the Cloud AI 100 export and compiler shaped the realization summarized in §5.

B.1 What the Compiler Does by Default

Slicing across SoCs. With its default heuristic the partitioner splits the down-projection weights of all 64 lanes four ways along the output dimension, so each SoC computes a 512-column slice of every expert’s output. This requires an all-gather of the intermediate activation (36 MiB per stage at $C = 128$) and a re-shard of the output (24 MiB per stage) before the combine; cross-SoC traffic reaches 121.6 MiB per layer, while gate and up projections stay expert-parallel.

A dense accumulator. The exported combine scatters each stage’s weighted expert rows into an accumulator of shape $[64, T, d]$, zero-filled by a single core, gathered and updated between the two stages, reduced over each SoC’s 16 lanes by one einsum that the compiler runs through DRAM on one core, and finally summed over the four SoCs on four cores of SoC 0. From $T = 256$ on, this last sum also spills into a DRAM-backed reduction on one core (0.84 ms). Every part scales with T ; none scales with the number of routed rows.

Lane counts must divide 16. The compiler maps a stage

onto a SoC’s cores by its lanes per SoC: counts that divide 16 map evenly, while in our lane-split programs 17–19 lanes split the GEMMs unevenly and 20–32 lanes collapsed onto four cores per SoC (3.7–4.3 ms for a stage that runs in 0.5 ms on 16 cores).

Stages run one after another. The compiler schedules a layer’s expert stages sequentially, in its own order; in our tiered programs a stage of L lanes per SoC ran one lane per core on L cores. Every additional stage adds a window of its own.

B.2 Expert-Parallel Compilation with an On-Device KV Cache

We compile with a weight-splitting degree of one across SoCs (`-mdts-mos=1`), which forbids cutting any weight matrix across SoCs. The lane axis is then the only way to distribute the batched expert GEMMs, and each SoC computes 16 whole experts per stage, one per core; the all-gather and re-shard of the down projection disappear, and the layer’s cross-SoC traffic falls from 121.6 to 1.5 MiB.

The setting is global and also stops the compiler from splitting the attention projections by KV head. The cache update then comes out token-sliced while its input is whole, the compiler cannot pair the 96 cache inputs with their outputs, and all 192 KV buffers become host I/O. We restore the pairing at the graph level with *head-parallel attention*: the four KV-head groups of eight query heads each (Ainslie et al., 2023) become a leading batch axis of every attention operator, with per-group weights of shapes $[4, 2048, 1024]$ for queries, $[4, 2048, 128]$ for keys and values and $[4, 1024, 2048]$ for the output projection, whose four partial outputs are summed at the end. Under expert-parallel compilation the batch axis lands one group per SoC, the cache operations of KV head h stay on SoC h , and the program again exposes no KV buffer; the regrouped output projection changes only the FP16 summation order.

B.3 Lowering the Combine

Dense-path rewrites. Before the combine is replaced, three rewrites remove serialized work from the exported one. (i) The first stage’s outputs were added to rows gathered from the freshly zeroed accumulator; we drop this read of zeros. (ii) The per-SoC sum over lanes, one einsum that the compiler ran through DRAM on one core, is cut into 16 tiles of $T/16$ tokens. (iii) The routing’s prefix sums over tokens, which give each assignment its row in the expert’s compact input and ran as a serial `CumSum` on one core, become Hillis–Steele scans of $\log_2 T$ shift-and-add steps (Hillis & Steele, 1986). All three leave the output bit-identical. They repair this export and this compiler’s lowering rather than change the combine’s formulation, and the evaluation keeps them as a separate step.

Token-owned combine. The device gather does not zero-fill out-of-range indices, so a safe index and a mask are both required, and the per-SoC work must stay on the lane axis the compiler already splits over SoCs: written as explicit per-SoC slices, the combine shipped expert outputs between SoCs (3 to 78 MiB) and doubled the layer’s device time.

Elementwise final sum. Written as 16 einsum tiles, the sum over the four partials ran on four cores per SoC; written as the elementwise tree $(P_0 + P_1) + (P_2 + P_3)$ per token tile, it spreads over all cores, and the compiler lowers it as a reduce-scatter that the next layer’s input normalization completes.

B.4 Lowering the Rank Sort and the Tiers

One gathered constant. The counts come from the routing mask, and the per-SoC sort is a TopK over each SoC’s 32 counts. Each tier’s three weight matrices come from Gather(bank, π) on one constant that holds all 128 experts, the only gather form this compiler keeps fast, lowered to indirect DMA (§3.3). The constant is replicated, so every SoC holds the weights of all 128 experts, 22.5 GiB per SoC instead of 6.7 GiB, within the SoC’s 32 GB; the algorithm itself needs only each SoC’s own 32 experts.

Off the critical path. The routing mask, the prefix scans and the slot tables are computed once over all 128 experts in native order, independent of the sort; only a row permutation of these tables and the weight gathers wait for it. Sorting first and running the routing on permuted columns makes everything wait (12 layers: +2.5 ms over capacity- T padding, against +1.6 ms). Gathered weights are also sensitive to the compiler’s maximum tile size: with the default, the first expert GEMM waits 0.085 ms for its first gathered tile, with 512 KiB tiles 0.032 ms. For the rank sort the best size is 512 KiB for $T \leq 256$ and 1024 KiB for $T = 512$ (Appendix E); other programs differ (Table 3).

Tier widths and count. Stages map evenly only when their lanes per SoC divide 16, run one after another, and use L cores for L lanes (Appendix B.1), so tiers take widths from $\{1, 2, 4, 8, 16\}$. At $T = 512$ the first tier is bound by the SoC’s shared memory traffic: eight lanes of 512 rows finish in 0.8–1.0 ms, sixteen in 2.2 ms. Three tiers of 8, 8 and 16 lanes balance these effects (§6.5).

B.5 Graph Transformations

RankTier is a set of ONNX graph transformations, about 1,100 lines of Python, applied to the Qwen3-30B-A3B graph that the vendor’s QEfficient library exports for Cloud AI 100 (Qualcomm Innovation Center, 2024). Each step of the ladder (§6.1) is a separate pass, so every intermediate program can be compiled and measured. The passes locate

operators by structure rather than by name (layer 0 numbers some nodes differently and folds a 64-lane constant into its stage), regenerate the T -dependent constants of the scans and tiles for each chunk length, and clone the stage subgraph once per tier with its own row order, weight gathers and capacity. A static If in the export that squeezes the token table is replaced by the equivalent Squeeze in every clone, and the combine’s map from a lane to (tier, SoC, row) becomes a set of small per-tier tables. The same pass builds the evaluation’s variants: the export’s own combine, fixed expert order through the gathers, fixed-size blocks, and tier capacities read from input lengths for the overflow fallback.

Each chunk length is its own program, with sequence length T and context $2T$, MXFP6 matrix weights, FP16 activations, 16 cores per SoC, depth-first scheduling, expert-parallel slicing and retained state (Appendix A). Static programs occupy 25 GiB, programs with gathered weights 88 GiB. Every program returns each lane’s token count in every layer.

C ROUTING CAPTURES AND TIER CALIBRATION

Routing captures run the model’s FP32 layers on the CPU and record every layer’s top-8 experts. At $T = 128$ we use 50 chat prompts per workload; at 512 tokens, 8 GSM8K chunks and 4 each from HumanEval, MMLU and SWE-bench Lite; the 256-token calibration set holds 16 GSM8K chunks and both halves of each 512-token chunk (56 in total). The held-out GSM8K chunks of the evaluation use test questions 1000–1003 (256 tokens) and 1004–1011 (512 tokens), outside the calibration range. Table 6 gives the largest per-SoC load at selected ranks, the input to the tier rule of §4.2.

Table 6. Largest load of the rank- r expert of a SoC over all calibration chunks, layers and SoCs.

T	$r = 1$	2	4	8	9	16	17	32
128	127	122	105	33	32	14	12	3
256	256	223	134	58	50	25	25	4
512	508	424	240	101	89	51	50	10

A fixed layout calibrated per workload fares worse the more its traffic varies. With 25 calibration prompts of the same workload, 64–92% of that workload’s held-out prompts overflow in some layer; calibrated on 175 prompts of the seven other workloads, 24–100% do. With 200 prompts from all eight, the fraction falls to 12–36%, and with random pooled subsets of 8, 32, 128 and 200 prompts to 100%, 70%, 27% and 21%. Under the per-SoC rank sort, ranks

17–32 need at most 12 rows at $T = 128$ in every layer of every workload.

D ADDITIONAL ONE-LAYER MEASUREMENTS

On a one-layer replay (layer 2, MXFP6, expert-parallel), the combine and the capacities interact. With capacity T in both stages, a token-centric predecessor of the token-owned combine saves 12%, 29% and 23% at $T = 128$, 256 and 512; fitted capacities save 6%, 14% and 13% with the dense combine but 9%, 15% and 25% once the combine is fixed, since the padded compute is then exposed. Both together are 20%, 40% and 42% faster than the naive layer.

E TILE SIZE OF GATHERED WEIGHTS

Table 7. Rank sort on 12-layer cuts: latency change against the token-owned program at capacity T (default tiles) for three maximum tile sizes.

T	Default tiles	512 KiB	1024 KiB
128	+4.7%	−1.4%	−
256	−5.1%	−18.2%	−7.9%
512	−12.7%	−9.3%	−15.9%

Tile size. Table 7 shows the sensitivity of gathered weights to the compiler’s maximum tile size. With default tiles the rank sort is 4.7% slower than the token-owned program at capacity T at $T = 128$; with 512 KiB tiles it is 1.4% faster. The best size moves with the chunk, 512 KiB up to 256 tokens and 1024 KiB at 512; static weights prefer the default only at $T = 128$ (small tiles slow them by 1–2%), while at 256 and 512 tokens 512 KiB tiles make the static program 8% and 16% faster (Table 3). Each program therefore uses its own best size. The tile size is the granularity of the compiler’s row skipping (Appendix G); at $T = 128$ a lane is one tile at every size.

F LADDER CHART

Figure 3 plots Table 2 relative to the production compilation.

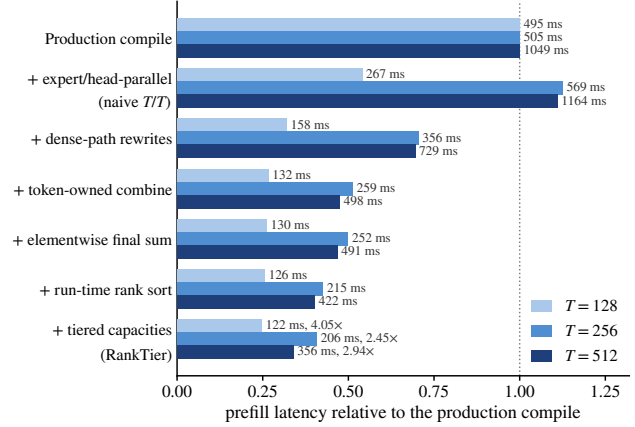


Figure 3. Prefill latency of the seven programs relative to the production compilation at the same chunk length, with the latency of each program and the speedup of RankTier.

G PROFILES OF THE ATTRIBUTION PROGRAMS

Table 8. Layer 1 of instrumented two-layer cuts at $T = 512$: MoE layer time, tensor-unit busy time summed over a SoC’s cores (busiest SoC) and MXFP6 weight bytes read per SoC, per stage in execution order.

Program	MoE	HMX busy (ms)	Weights (MiB)
A, default tiles	6.87	26.0 / 21.1	53 / 56
A, 512 KiB	4.12	5.1 / 3.1	56 / 56
B, 1024 KiB	5.23	18.5 / 10.7	56 / 56
C, 1024 KiB	5.74	24.3 / 8.9	56 / 56
C, 512 KiB	6.45	4.2 / 1.8	56 / 56
D, 1024 KiB	5.07	27.3 / 0.6	56 / 49
E (RankTier)	3.65	0.8 / 3.3 / 0.35	49 / 28 / 28
Blocks, 64, dropless	12.6	31.4	337.5
Blocks, 64, trace-specialized	8.5	9.9	168.8

Tile predication. The export ends each expert lane with $\text{Where}(r < n_e, y, 0)$, where n_e is the expert’s run-time token count. The compiler lowers it as a range select that writes the zero rows directly and gates the work producing the others: it cuts the lane’s rows into tiles, evaluates for each tile at run time whether it holds a row below n_e (a compare and a reduction to a scalar), and runs the tile’s activation gather, GEMMs and SiLU only if so. A lane with n tokens and row tile R therefore computes $R \lceil n/R \rceil$ rows, which holds exactly for every lane of every stage and SoC of the programs above: a 512-row lane is one tile at the default ($R = 512$), two at 1024 KiB and four at 512 KiB, and a lane spread over two cores is cut per core (RankTier’s 128-row lanes: $R = 64$). Per SoC, A computes 13.8–15.4k of its 16.4k rows at the default and 3.5–3.8k at 512 KiB, B and C 6.9–7.7k at 1024 KiB, and RankTier 3.3–3.5k of 6.1k.

An expert’s weight load and dequantization feed all of its lane’s tiles: for an empty expert they are skipped only when the lane is one tile (A at the default, the 64-row tiers of D and RankTier) and run otherwise. At 512 KiB the gathered program C computes 128-row tiles like A but spills the activations of its GEMMs to DRAM (95–97 MiB written and read back per SoC, against none for A), which lengthens its stages. Both rules are read off the traces; the compiler does not document them.

H BLOCK SCHEDULING BY BLOCK SIZE

Table 9. Block scheduling against RankTier on two-layer cuts at $T = 512$ (ms; RankTier 15.9). Blocks per SoC in parentheses.

B	dropless	calibrated	trace-specialized
16	38.3 (288)	28.0 (176)	19.7 (96)
32	30.3 (160)	21.2 (96)	18.1 (64)
64	24.6 (96)	20.1 (64)	17.9 (48)
128	24.0 (64) = dropless		18.1 (32)
vs. RankTier	$1.51\times$	$1.26\times$	$1.12\times$

I REJECTED ALTERNATIVES

Table 10 lists the alternatives we rejected. Splitting a loaded expert over two lanes halves the first tier’s GEMM time in isolation, but the compiler keeps the 16 lanes on eight cores per SoC and dequantizes each expert twice, so the program gets 6% slower. Co-locating experts that fire together cuts the SoCs a token touches from 3.63 to 2.42, but it doubles the order statistics the lower tiers must cover (at $T = 512$ from 50 to 85 rows at rank 17) and targets an exchange that is only 3% of the model.

Table 10. Alternatives we built or analyzed and rejected ($T = 512$ unless noted).

Alternative	Result	Reason
Sort first, then route on permuted columns	+2.5 vs. +1.6 ms (12 layers, $T=128$)	all routing waits for the sort
Row-chunked expert GEMMs	+25%, +66% (one layer)	each chunk re-streams and re-dequantizes the weights
One partition per SoC	$1.7\times$ slower	the runtime runs partitions in sequence
Per-layer first-tier capacity	−10–15% of its rows	held-out chunks overflow in 1–5 layers
Co-locate experts that fire together	$3.63 \rightarrow 2.42$ SoCs per token	doubles lower-tier loads; the exchange is 3%
Split first-tier experts over two lanes	+6% (two layers)	16 lanes stay on 8 cores; $1.8\times$ dequantization
Four tiers	−8.6% vs. −14.1%	one more sequential window
No depth-first scheduling	+1–2%	stages still run in sequence

J STAGE WINDOWS

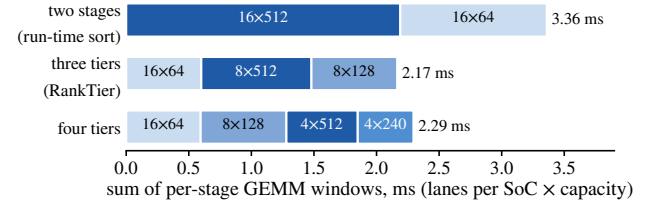


Figure 4. GEMM window of each expert stage in execution order at $T = 512$, two-layer profiles; segments are the per-stage medians over SoCs, labeled with lanes per SoC $\times$ capacity.

K ACCURACY PER PROGRAM

Table 11. Relative L_2 error of the last-position logits against FP32, and the next token ($\checkmark$: matches FP32).

Program	$T = 128$	$T = 256$	$T = 512$
1 Production	0.188 $\checkmark$	0.089 $\times$	0.095 $\checkmark$
2–3 Naive T/T , rewrites	0.172 $\checkmark$	0.077 $\checkmark$	0.094 $\checkmark$
4 Token-owned combine	0.172 $\checkmark$	0.073 $\checkmark$	0.092 $\checkmark$
5 Elementwise sum	0.190 $\checkmark$	0.077 $\checkmark$	0.095 $\checkmark$
6–7 Rank sort, RankTier	0.170 $\checkmark$	0.087 $\times$	0.088 $\checkmark$